Data Analytics Project: IGN Game Reviews Analysis

Dataset Used: ign.csv

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyze game review scores, platform popularity, and genre distribution over time using the `ign.csv` dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (🟢 Easy)

Description:

The dataset is loaded using Pandas and explored with `head()`, `tail()`, and `shape`. The unnecessary **"Unnamed: 0"** column is removed. Missing values in `score`, `genre`, and `platform` are checked and handled by filling `score` with mean and `genre` with mode. Data types are corrected by converting `score` to float and date columns to integers. The dataset is then clean and ready for analysis.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
4.	Check for missing values in `score`, `genre`, and `platform`.
5.	Handle missing values: Fill the numeric column `score` with the mean, and the categorical column `genre` with the mode.
6.	Ensure correct data types: Convert `score` → float and `release_year`, `release_month`, `release_day` → integer.

Outputs:

```
First 5 rows are:
   Unnamed: 0 score_phrase  ... release_month release_day
0           0      Amazing  ...             9          12
1           1      Amazing  ...             9          12
2           2        Great  ...             9          12
3           3        Great  ...             9          11
4           4        Great  ...             9          11

[5 rows x 11 columns]
```

1.First 5 rows of dataset

```
Last 5 rows:
       Unnamed: 0 score_phrase                                title  ... release_year release_month  release_day
18620       18620         Good               Tokyo Mirage Sessions #FE  ...         2016             6           29
18621       18621      Amazing        LEGO Star Wars: The Force Awakens  ...         2016             6           29
18622       18622     Mediocre  Star Ocean: Integrity and Faithlessness  ...         2016             6           28
18623       18623  Masterpiece                                 Inside  ...         2016             6           28
18624       18624  Masterpiece                                 Inside  ...         2016             6           28

[5 rows x 11 columns]
```

2. Last 5 rows of dataset

```
Shape of dataset is:
(18625, 11)
```

3. Shape of dataset

Conclusion:

Scenario 1 successfully prepares the dataset by cleaning and organizing the data. Unnecessary columns are removed, missing values are handled, and data types are corrected. These steps ensure the dataset is accurate, consistent, and suitable for further analysis or modeling.

Scenario 2: Line Graph (Score Trend) + Save (🟢 Easy)

Description:

This scenario analyzes how game scores change over time by grouping the data by `release_year` and calculating the average score for each year. The results are converted into NumPy arrays for plotting. A line graph is created with years on the X-axis and average scores on the Y-axis. The graph is labeled with a title and axis labels for clarity. Finally, the plot is saved as **"avg_score_trend.png"**.

Task	Description
1.	Group data by `release_year`.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with `release_year` on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: `plt.savefig("avg_score_trend.png")`.

Graph:

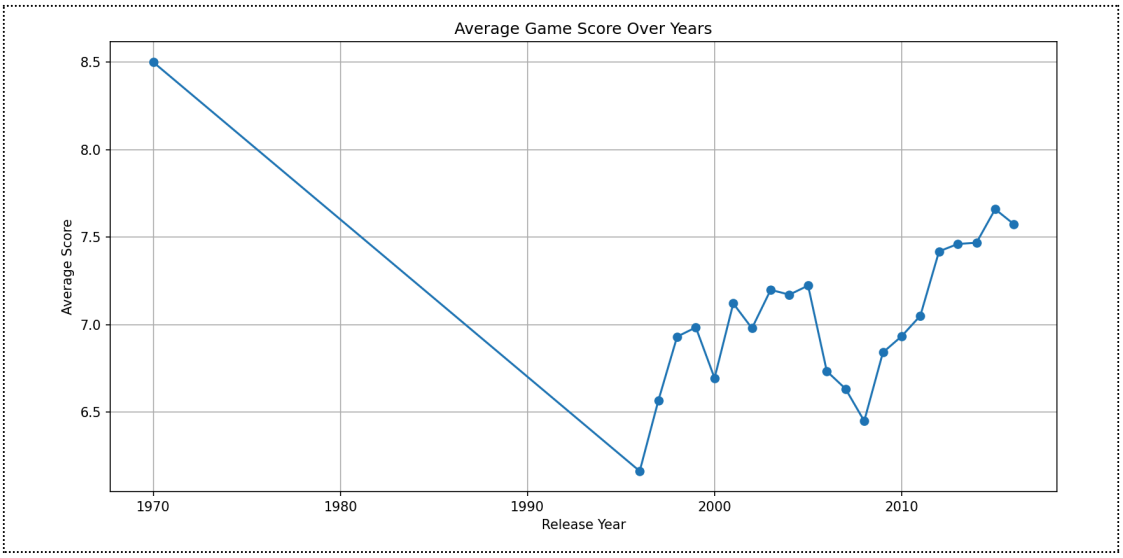

Fig 1. Average Game Score Over Years

Conclusion:

Scenario 2 not only visualizes the trend of average game scores over the years but also helps identify patterns such as improvements, declines, or consistency in game quality. The use of aggregation simplifies large data into meaningful insights, making year-wise comparisons easier. Converting data into NumPy arrays ensures efficient plotting, while proper labeling improves readability. Saving the graph allows for easy sharing and future reference. Overall, this step transforms raw data into clear, interpretable visual insights.

Scenario 3: Filtering + Bar Chart + Save (🟡 Medium)

Description:

This scenario focuses on comparing top gaming platforms based on high-rated games. The dataset is filtered to include only games with a `score` greater than 7. The number of such games is counted for each platform, and the top 10 platforms are selected using Pandas. The results are converted into NumPy arrays for visualization. A bar chart is plotted with platforms on the X-axis and game counts on the Y-axis, with rotated labels for better readability, and the graph is saved as **"top_platforms_bar.png"**.

Task	Description
1.	Filter the dataset to include only games where `score > 7`.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with `platform` on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: `plt.savefig("top_platforms_bar.png")`.

Output:

```
dataset where score is > 7:
        Unnamed: 0 score_phrase  ... release_month release_day
0                0      Amazing  ...             9          12
1                1      Amazing  ...             9          12
2                2        Great  ...             9          12
3                3        Great  ...             9          11
4                4        Great  ...             9          11
...            ...          ...  ...           ...         ...
18619        18619         Good  ...             7          28
18620        18620         Good  ...             6          29
18621        18621      Amazing  ...             6          29
18623        18623  Masterpiece  ...             6          28
18624        18624  Masterpiece  ...             6          28

[9800 rows x 11 columns]
```

4. Filtered dataset

Graph:

The bar chart compares the top 10 gaming platforms based on the number of games with a score above 7. Each bar represents a platform, and its height shows the count of high-rated games. Platforms with higher bars indicate better performance in producing quality games. The visualization makes it easy to identify the most successful platforms at a glance.

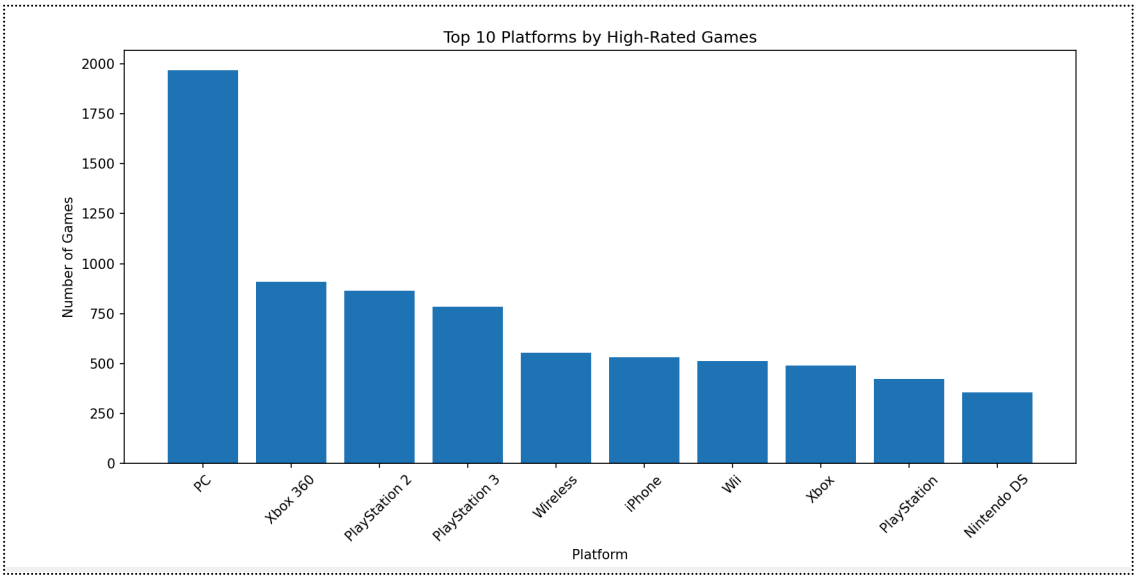

Fig 2. Top 10 Platforms by High-Rated Games

Conclusion:

Scenario 3 helps identify the most successful gaming platforms by focusing on high-rated games. Filtering the dataset highlights only quality games, making the analysis more meaningful. The bar chart clearly compares platforms, showing which ones consistently produce better-rated games. This visualization simplifies decision-making and provides useful insights into platform performance.

Scenario 4: Aggregation + Pie Chart + Save (🟡 Medium)

Description:

This scenario analyzes the distribution of game genres by counting the number of games in each genre. The top 5 genres are selected using Pandas for better clarity. The data is prepared into labels and values for visualization. A pie chart is created showing each genre's share with percentage labels using `autopct`. Finally, the chart is saved as **"genre_distribution.png"**.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

Output:

```
Top 5 genres are:
 genre
Action        3797
Sports        1916
Shooter       1610
Racing        1228
Adventure     1175
Name: count, dtype: int64
```

5.Displaying top 5 genres

Graph:

The pie chart shows the distribution of the top 5 game genres based on the number of games in each category. Each slice represents a genre, and its size indicates its share of the total games. The percentage labels make it easy to understand the proportion of each genre. Larger slices show more popular genres in the dataset. Overall, the graph provides a clear view of genre popularity at a glance.

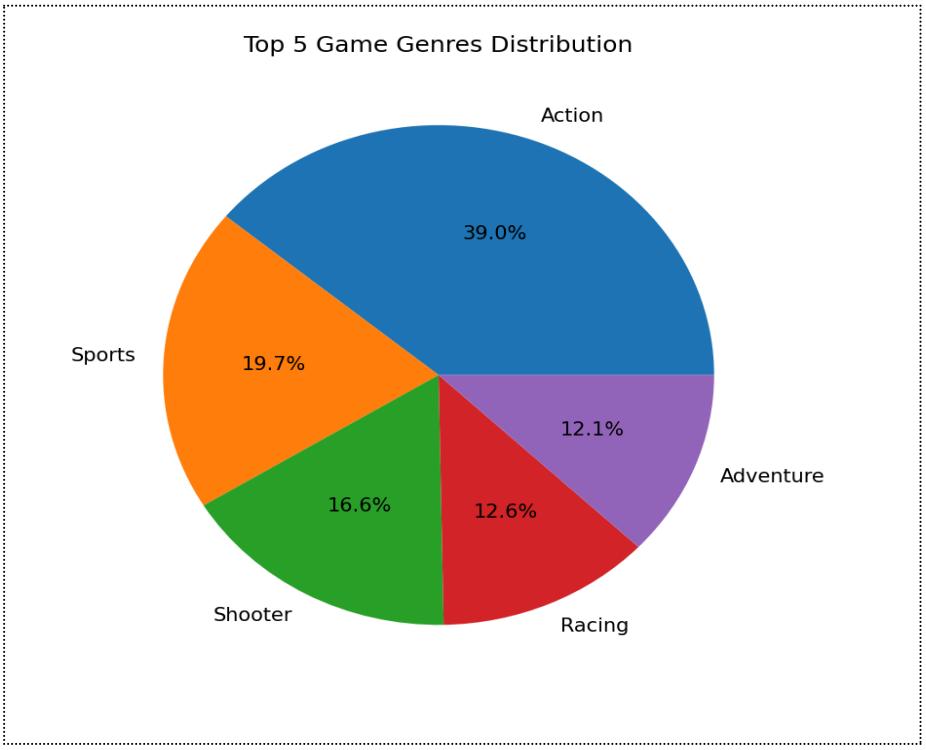

Fig 3. Top 5 Game Genres Distribution

Conclusion:

Scenario 4 provides a clear understanding of how games are distributed across different genres. By focusing on the top 5 genres, the analysis highlights the most popular categories in the dataset. The pie chart makes it easy to visualize proportions and compare genre popularity. This helps in identifying dominant genres and understanding user preferences in gaming.

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:

This scenario performs advanced analysis by creating a **score_category** feature and converting `editors_choice` into numeric form. NumPy is used to calculate yearly score growth using average scores. Multiple visualizations are created, including a line graph, stacked bar chart, and histogram to show trends and distributions. All graphs are saved for documentation. Finally, insights are drawn on score trends, peak years, and the relationship between editor's choice and high scores.

Task	Description
1.	Create a new column `score_category`: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the `editors_choice` column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's `np.diff()` to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per `release_year`.
5.	Plot a **Stacked Bar Chart** showing the count of `score_category` per `release_year`.
6.	Plot a **Histogram** showing the distribution of `score`.
7.	Save the graphs: `plt.savefig("score_trend.png")`, `plt.savefig("score_category_stacked.png")`, `plt.savefig("score_distribution.png")`.
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if `editors_choice` correlates with high scores.

Output:

```
   score score_category  editors_choice
0    9.0       Excellent               1
1    9.0       Excellent               1
2    8.5            Good               0
3    8.5            Good               0
4    8.5            Good               0
```

5. Created a new column

Graph:

- **Line Graph :** The line graph in this scenario shows the trend of average game scores over different release years. It helps visualize how game quality has changed over time. Rising points indicate improvement in scores, while drops show decline in certain years. This makes it easy to identify patterns or peak performance periods in game reviews.

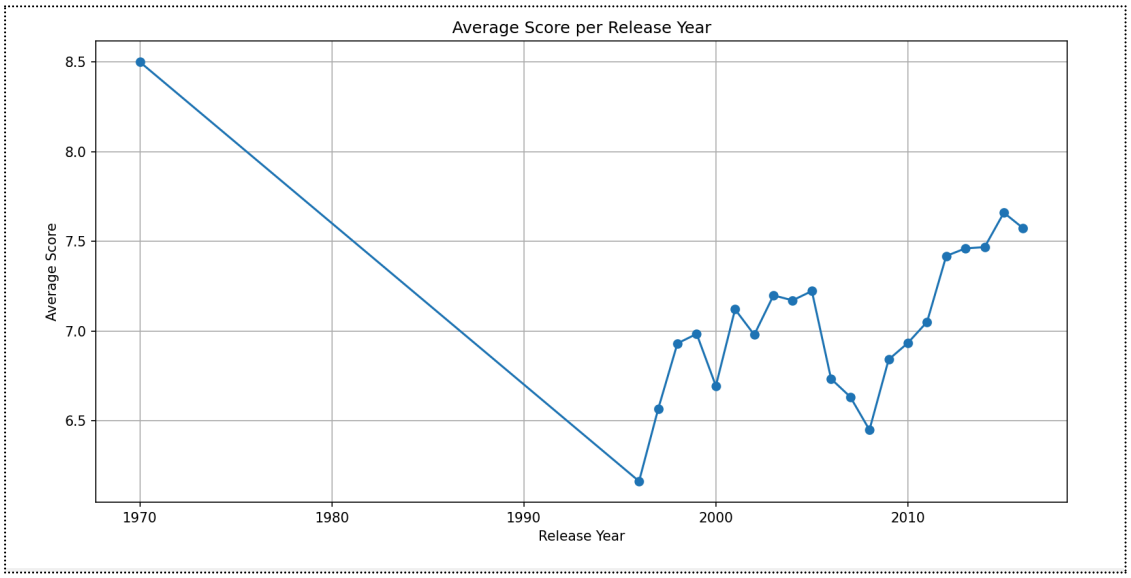

Fig 4. Average Score per Release Year

- **Stacked Bar Chart:** The stacked bar chart shows the number of games in each score category (Excellent, Good, Average) for every release year. Each bar represents a year, and the segments within it show how scores are distributed across categories. This helps compare not only total games per year but also the quality breakdown. It clearly shows how game quality changes over time in different score groups.

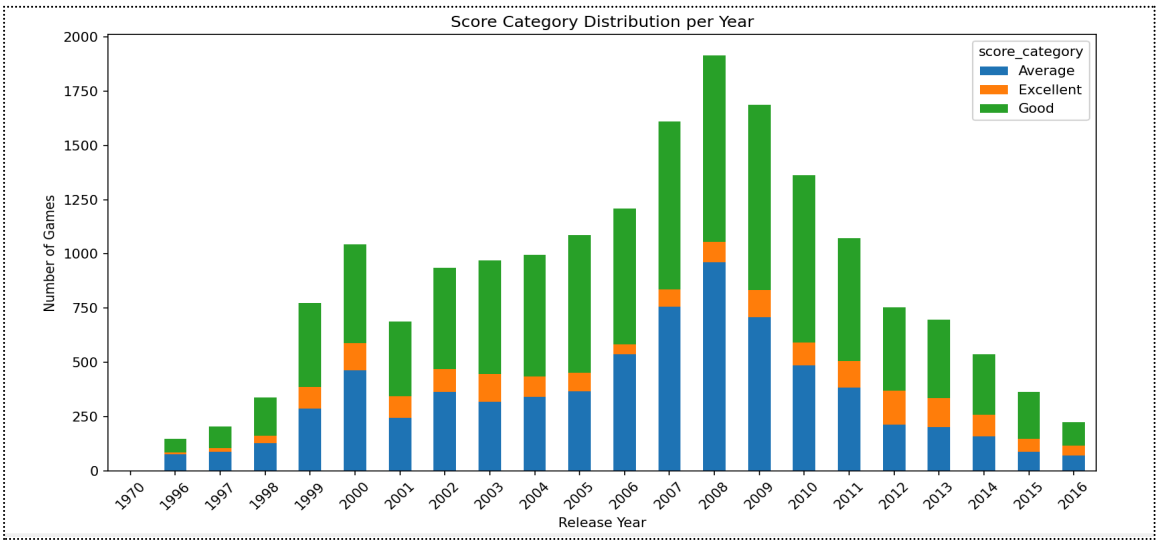

Fig 5. Score Category Distribution per Year

- **Histogram:** The histogram shows the distribution of game scores across the dataset. It groups scores into intervals (bins) and displays how many games fall into each range. This helps identify whether most games are low, medium, or high rated. It also highlights the overall pattern of score spread, such as skewness or concentration around certain values.

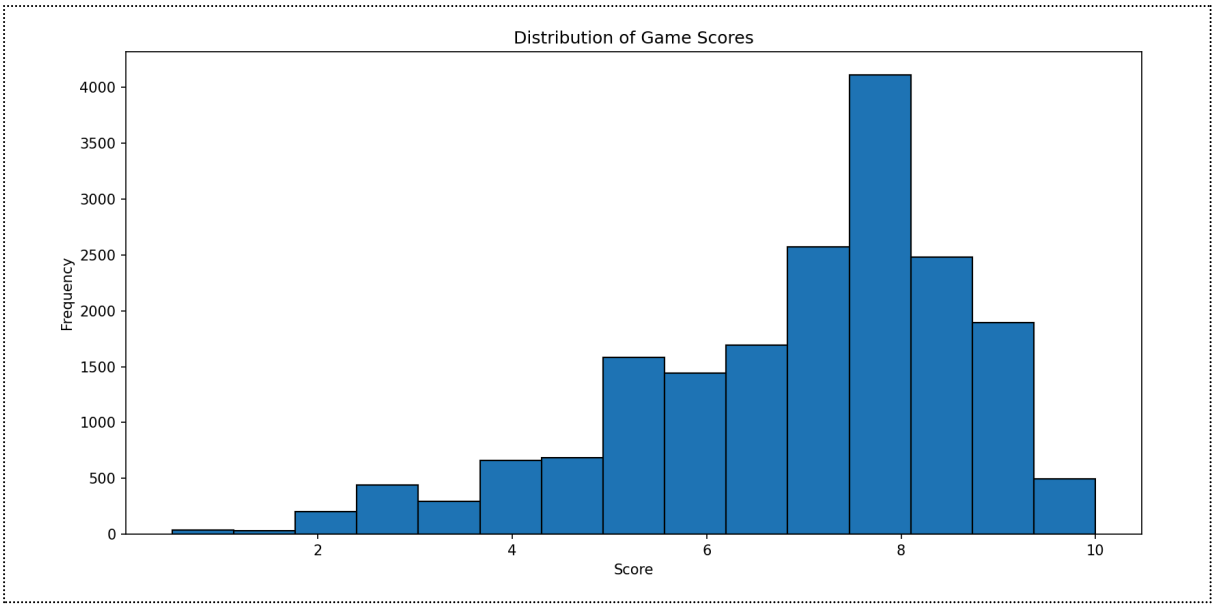

Fig 6. Distribution of Game Scores

Conclusion:

Scenario 5 provides a comprehensive analysis of game review patterns using feature engineering, NumPy calculations, and multiple visualizations. It reveals how game scores are distributed, how they change over time, and how categories like Excellent, Good, and Average vary across years. The analysis also helps identify peak performance years and trends in score growth. Additionally, it gives insight into whether an editor's choices are linked to higher scores. Overall, this scenario delivers a deep understanding of game quality trends in the dataset.

Insights:

- **Which years had the highest scores?**
  The highest average score is observed in 1970 (8.5). Some later years like 2001 and 2005 also show relatively higher average scores compared to other years in the dataset.
- **Whether high scores increased over time?**
  No, high scores do not show a consistent increase over time. The trend fluctuates across years, with both increases and decreases, rather than a steady upward pattern.
- **If editors_choice correlates with high scores?**
  Games marked as `editors_choice = 1` generally have higher average scores compared to those marked as `0`, indicating a positive correlation between editor's choice and higher ratings.

Final Conclusion:

This project focused on analyzing the IGN game reviews dataset through proper data preprocessing, feature engineering, and visualization techniques. Missing values were handled, data types were corrected, and new features were created for better analysis. Various graphs such as line charts, bar charts, pie charts, stacked bar charts, and histograms helped in understanding trends and distributions. The analysis highlighted score trends over time, top-performing platforms, and popular genres. Overall, it successfully converted raw data into meaningful insights about game reviews and ratings.

What you have learned

Through this project, I learned how to work with real datasets using Python. I understood how to clean data by fixing missing values, removing unnecessary columns, and correcting data types so the data becomes usable. I also learned how to create new features to make analysis more meaningful. By using graphs like line charts, bar charts, pie charts, and histograms, I was able to clearly see trends and patterns in the data. Overall, this project helped me understand how raw data can be turned into useful insights step by step.